

Hardware-Accelerated Drawing Plugin — Vendor Brief

Reference repository: `https://github.com/vritravaibhav/ha_plugin_example`
Reference plugin: `plugins/deltav100` (Nomivision Delta V100 motherboard)
Document version: 1.0 · Date: 22 May 2026

1. Objective

We need a Flutter plugin that drives **your hardware-accelerated drawing panel** (your motherboard / SDK) and exposes **exactly the same Flutter-side API** as our existing `deltav100` plugin. Our host application uses a single widget — `HardwareAcceleratedCanvas` — to drive the canvas. That widget, its constructor parameters, callbacks, view-type identifier, and channel names must remain identical so the host app can switch panels by only swapping the plugin dependency. The native (Android) implementation is yours; the Flutter contract is fixed.

2. Deliverables

1. Source code	A public or shared GitHub repository , forked from <code>https://github.com/vritravaibhav/ha_plugin_example</code> , containing your plugin under <code>plugins/<your_plugin_name>/</code> .
2. Working APK	A signed debug or release APK built from that fork that runs on your target hardware and demonstrates the canvas working end-to-end.
3. Build instructions	A short <code>README.md</code> in the plugin folder with build steps, target Android API level, any required system services, and licensing notes for any proprietary SDK / .so libraries you bundle.

Mandatory: the APK code must be a fork of our repository. Do not start from a fresh Flutter project. We need git history that shows the diff between your plugin and our reference plugin.

3. Flutter-side contract (do NOT change)

3.1 Widget — `HardwareAcceleratedCanvas`

This is the only widget the host app instantiates. Class name, parameter names, types, and order must match exactly.

```

class HardwareAcceleratedCanvas extends StatefulWidget {
  const HardwareAcceleratedCanvas({
    super.key,
    required this.penColor,
    required this.isRainbow,
    required this.isDashed,
    required this.strokeWidth,
    required this.isHighlight,
    required this.isMultiFingerEnabled,
    this.onStrokeStarted,
    this.onStrokeCommitted,
    this.gestureRecognizers,
  });

  final Color penColor;
  final bool isRainbow;
  final bool isDashed;
  final double strokeWidth;
  final bool isHighlight;
  final bool isMultiFingerEnabled;
  final VoidCallback? onStrokeStarted;
  final Deltav100StrokeCallback? onStrokeCommitted; // signature below
  final Set<Factory<OneSequenceGestureRecognizer>>? gestureRecognizers;
}

typedef Deltav100StrokeCallback = void Function(Map<String, dynamic> data);

```

You may rename the typedef (e.g. `YourPanelStrokeCallback`) but the **function signature must stay** `void Function(Map<String, dynamic> data)` and the widget property must remain `onStrokeCommitted` .

3.2 Parameter semantics

Parameter	Type	Meaning
<code>penColor</code>	Color	Current pen colour (ARGB int passed to native).
<code>isRainbow</code>	bool	Rainbow gradient stroke shader.
<code>isDashed</code>	bool	Dashed stroke pattern.
<code>strokeWidth</code>	double	Pen width in logical pixels.
<code>isHighlight</code>	bool	Translucent highlighter mode.
<code>isMultiFingerEnabled</code>	bool	Allow simultaneous multi-touch strokes.
<code>onStrokeStarted</code>	VoidCallback?	Fires once when the first pointer goes down.
<code>onStrokeCommitted</code>	callback	Fires on pointer up with the full stroke payload (see 3.4).
<code>gestureRecognizers</code>	set	Passed through to the native <code>AndroidView</code> .

3.3 Platform-view & channel identifiers (must match)

Constant	Required value
Android view type	<code>fb_drawing_view_deltav100</code>
Stroke event channel	<code>com.demopaint.draw_app/hardware_stroke</code>
Framebuffer method channel	<code>com.demopaint.draw_app/framebuffer</code>

If you absolutely must rename your view type internally, you may — but the Flutter package must continue to register the platform view under `fb_drawing_view_deltav100` so the host app code is unchanged.

3.4 `onStrokeCommitted` payload schema

The `Map<String, dynamic>` delivered to `onStrokeCommitted` on every stroke **must** contain these keys with these types (matches what `Deltav100StrokeBridge.java` emits today):

```
{
  "points":      List<Map>,    // each item: { "x": double, "y": double } in view-
space px
  "color":       int,          // ARGB
  "strokeWidth": double,      // logical px
  "dashed":      bool,
  "rainbow":     bool,
  "viewWidth":   double,      // native view width  (px)
  "viewHeight":  double,      // native view height (px)
  "pointer":     int           // Android pointerId
}
```

Additionally, your event stream must emit these two control messages exactly as the reference does:

```
{ "strokeStarted": true } // when first finger goes down
{ "requestBrushSync": true } // when native surface (re)creates and needs brush
state
```

3.5 MethodChannel surface (must match)

The framebuffer method channel must accept these methods with these argument shapes:

Method	Arguments
setDrawColor	{ color: int /* ARGB */ }
setEraserMode	{ enable: bool }
setStrokeSize	{ size: double }
setEraserSize	{ size: double }
setDashedStroke	{ enable: bool }
setRainbowMode	{ enable: bool }
setHighlightMode	{ enable: bool }
clearNativeCanvas	—

3.6 Host-app usage that must compile unchanged

After dropping in your plugin (only the `pubspec.yaml` dependency path changes), this example app file must build and run without edits:

```
import 'package:<your_plugin>/<your_plugin>.dart';
import 'package:flutter/gestures.dart';
import 'package:flutter/material.dart';

HardwareAcceleratedCanvas(
  penColor: Colors.black,
  isRainbow: false,
  isDashed: false,
  strokeWidth: 20,
  isHighlight: false,
  isMultiFingerEnabled: true,
  onStrokeStarted: () { /* ... */ },
  onStrokeCommitted: (data) { /* ... */ },
  gestureRecognizers: <Factory<OneSequenceGestureRecognizer>>{
    Factory<OneSequenceGestureRecognizer>(() => EagerGestureRecognizer()),
  },
)
```

4. Native side — what we expect (informational only)

You are **free to use your own native SDK / framebuffer driver / motherboard APIs**. We do not require you to copy our Nomivision `WhiteBoardSpeedup` integration. The only constraints on the native side are the channel + view-type identifiers in §3.3 and the payload schema in §3.4.

For reference, our existing Android implementation is split into:

- `Deltav100Plugin.java` — channel + factory registration
- `Deltav100FbDrawingViewFactory.java` — `PlatformViewFactory`
- `Deltav100FbDrawingView.java` — `SurfaceView` that draws into the FB
- `Deltav100HardwareAccel.java` — vendor SDK wrapper
- `Deltav100StrokeBridge.java` — emits stroke events to Flutter
- `Deltav100Config.java` — screen dimensions

You may mirror this layout or use any structure you prefer.

5. Functional acceptance criteria

1. The example app (`lib/main.dart` in the fork) builds and installs on your target device with no edits beyond the plugin `path:` in `pubspec.yaml`.
2. Touching the screen draws strokes with hardware acceleration (no Flutter-level rasterisation).
3. Stroke width, colour, dashed, rainbow, highlight, and eraser modes all visibly work and respond to `MethodChannel` calls from Flutter.
4. Multi-finger drawing works when `isMultiFingerEnabled = true`.
5. `onStrokeStarted` fires on first pointer down.
6. `onStrokeCommitted` fires on pointer up with the payload schema in §3.4.
7. The native FB is wiped within ~300 ms after pointer-up so the Flutter side can take over rendering seamlessly (or document your equivalent handoff behaviour).
8. `clearNativeCanvas` clears the panel.

6. Handover checklist

- GitHub URL of the fork (public, or invite our account as collaborator).
- APK file (debug or signed release) built from the latest commit on that fork.
- Commit SHA the APK was built from.
- Target panel / motherboard model + Android version verified on.
- Any AAR / SO / proprietary dependencies and their licenses.
- One short demo video (15-30 s) showing the example app running on the panel.

7. Out of scope

- iOS / desktop builds (the widget returns `SizeBox.shrink()` off-Android — keep that).
- UI of the host app — we own `lib/main.dart` ; do not redesign it.
- Persistent storage, networking, account systems.

8. Contact

For any clarifications about the Flutter contract, open an issue on the source repository (https://github.com/vritravaibhav/ha_plugin_example) or reply to the email this brief was attached to.

Hardware-Accelerated Drawing Plugin — Vendor Brief · v1.0 · 22 May 2026

Reference: `plugins/deltav100` in `vritravaibhav/ha_plugin_example`